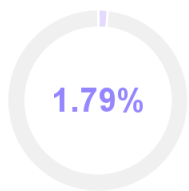


NO. 43fedac9a71282f6 | 2026-04-29 18:10:51

- 题目：基于Python的售楼管理系统的设计与实现
- 作者：周杭
- 检测所属单位：普洱学院

📄 论文字符数：29084 📖 论文页数：44 📊 表格数量：20 🖼️ 图片数量：22

检测结果



1.79%

疑似AIGC生成占比

98.21%

人工撰写占比

结果分布

序号	章节	疑似AIGC生成 文字/章节总字数	疑似AIGC生成 章节占比	人工撰写占比
1	摘要	0/566	0.0%	100.0%
2	Abstract	56/354	15.0%	85.0%
3	1.概述	259/2902	8.0%	92.0%
4	2.系统开发相关技术	0/3328	0.0%	100.0%
5	3.系统分析	0/2228	0.0%	100.0%
6	4.系统设计	0/3100	0.0%	100.0%
7	5.系统实现	0/3913	0.0%	100.0%
8	6.系统测试	0/2228	0.0%	100.0%

*注:格式规范的情况下可准确识别章节，若论文中无章节，可能会识别有误。

片段分布

■ 疑似AIGC生成 ■ 人工撰写



文字标注

■ 疑似AIGC生成 ■ 人工撰写

基于Python的售楼管理系统的设计与实现

普洱学院

学士学位毕业论文（设计）

题目 基于Python的售楼管理系统的设计与实现

学生姓名 周杭 学号 206021152

导师姓名 马航超 职称 未定级

学 院 理工学院

专 业 信息管理与信息系统

2026 年 4 月

学位论文原创性声明

本人郑重声明：所呈交的学位论文是本人在论文指导老师的指导下取得的研究成果。除文中已经注明引用的内容外，本学位论文的研究成果不包含任何他人创作的、已公开发表或没有公开发表的作品的内容，不存在AI人工智能论文代写情况。对本毕业论文（设计）的研究做出贡献的个人和集体，均已在文中以明确方式标明。本学位论文原创性声明的法律責任由本人承担。

学位论文作者签字：

年 月 日

学位论文使用授权书

本人完全了解并同意普洱学院拥有在《著作权法》规定范围内的学位论文使用权，即：（1）本毕业论文（设计）成果归普洱学院所有；（2）学位获得者必须按规定提交学位论文（包括纸质印刷本及电子版）；（3）学校图书馆和档案馆保存学位论文电子版和纸质版；（4）根据教育部有关规定，普洱学院向教育部指定单位提交公开的学位论文；（5）本人保留在其他媒体发布论文的权利。

本人遵循普洱学院有关毕业论文（设计）的相关规定，提交毕业论文（设计）的印刷本及相应电子版。本人同意普洱学院有权保存毕业论文（设计）的印刷本和电子版，并提供目录检索与阅览服务；可以采用影印、缩印、数字化或其他复制手段保存论文；在不以营利为目的的前提下，可以公布非涉密毕业论文（设计）的部分或全部内容。

本人承诺：本人的学位论文是在普洱学院学习期间创作完成的作品，并已通过论文答辩；提交的学位论文电子版与纸质版内容一致，如因不同造成不良后果由本人自负。

（保密的学位论文在解密后使用本授权书，本论文：☐不保密；☐保密 期
至 年 月止）

学位论文作者暨授权人签字：

年 月 日

摘 要

在房地产行业数字化转型的背景下，中小型房企传统售楼管理模式存在效率低、数据脱节、流程不规范等痛点。本文设计并实现了一款基Pytho的售楼管理系统，采用前后端分离架构，以Python+Flask为后端开发框架、Vue为前端核心技术、MySQL为关系型数据库，结合JWT完成身份认证与权限管控。

系统有六个主要的服务对象，即房产顾问、销售经理、销售控专员、会计、市场总监和客户本人。系统分为后台管理端和客户端两大部分。针对后台管理创建起六个核心模块：客户管理、房源管理、销售业务、财务收款、数据统计、权限设置。客户端可以进行在线看房、选房锁房以及进度查询，还可以在平台上办理相关的手续。可以说整个房产销售流程都串起来了，从跟进客户、房源管控、申请折扣到签约、盖章最后收款，没有一个环节被落下。但是实际跑下来发现这套系统确实能够帮助销售流程实现线上化，数据管理也更加规范，职能权限还做了非常细致的优化。传统的房地产销售行业存在的各种问题，比如撞单、房源乱糟糟、流程不透明等都可以得到解决。测试过程中还了解到公司内部运营效率得到了有效的提高，并且客户从看房到买完房的过程也变得更加顺畅了。

总体而言这套系统比较实用，上手也比较容易。功能设计上符合中小房企的实际需求。因此如果要想实现销售管理的数字化转型，这套系统可以提供一套靠得住的技术支持。

关键词：售楼管理系统；Flask框架；前后端分离

Abstract

In the context of digital transformation in the real estate industry, small and medium-sized developers still rely on old-fashioned sales management models. The result? Low efficiency,

disconnected data, and messy processes. So we wrote this article to design and actually roll out a proper sales management system.

On the tech side, we went with a front-end or back-end separation approach. The back-end uses Python + Flask as the framework, the front-end core is built with Vue, MySQL handles the relational database, and we use JWT for identity authentication and permission control. This system specifically serves six types of roles: real estate consultant, sales manager, sales supervisor, accountant, marketing director, and of course the customer.

The system is split into two main parts: the admin back-end and the client side. On the back-end, we built six core modules customer management, property management, sales operations, financial collections, data statistics & analysis, and permission settings. On the client side, users can view properties online, select and lock units, check progress, and even handle the paperwork over the internet. You could say the entire property sales process is now fully connected from following up with leads, managing inventory, requesting discounts, all the way to signing the contract, stamping it, and finally receiving the payment. No step gets left out.

When we actually tested it, we found the system does a real job of moving sales processes online. Data management becomes more standardized, and functional permissions are optimized in quite a detailed way. Those headaches that plague traditional real estate sales like order conflicts, messy inventory, and a lack of transparency the system handles them cleanly. We also noticed during our tests that internal operational efficiency improves noticeably, and on top of that, the customer experience becomes much smoother, from the first property visit all the way to closing the deal. Overall, this system is very practical and easy to pick up. Looking at the functional design, we feel it fits the real needs of small and medium-sized developers. So if you're looking to go digital with your sales management, this system can serve as reliable technical support.

Key words:Real estate sales management system;Flask framework;Separation of front-end and back-end

目 录

摘 要I

AbstractII

1. 概述1

1.1 研究背景与意义1

1.2 国内外研究现状1

1.3 研究内容和目标2

2. 系统开发相关技术4

2.1 Python技术4

2.2 Flask框架4	
2.3 Vue框架4	
2.4 MySQL数据库5	
2.5前后端分离框架5	
3. 系统分析7	
3.1 可行性分析7	
3.2 系统流程分析7	
3.2. 系统流程分析8	
4. 系统设计10	
4.1总体架构设计10	
4.2功能模块设计10	
4.3 角色功能设计11	
4.4 数据库设计12	
5. 系统实现15	
5.1 后端核心功能实现15	
5.2 前端界面实现19	
5.3 关键技术与难点解决26	
6. 系统测试28	
6.1 测试环境与测试方法28	
6.2 功能测试用例与结果分析28	
6.3 性能测试与优化措施29	
7. 总结与展望31	
7.1 总结31	
7.2 系统不足31	
7.3 展望31	
参考文献32	
附 录34	
致 谢37	

1. 概述

1.1 研究背景与意义

近些年来，数字经济在各个行业不断深入发展的同时也促使房地产行业由粗放式的传统管理转向精细化的运营[1][2]。但是我们发现很多中小型房企在销售管理方面还存在着明显不足，主要依靠手工记账、Excel表格，或者使用功能较弱的老软件来完成工作，并且各部门之间数据难以有效衔接。客户跟进信息分散于各个销售人员手中，房

源状态更新不及时，销售流程出现断点，管理层想要得到准确的数据进行决策也十分困难。这些问题不但会拖慢运营效率，还会对客户的资源整合以及销售流程的规范管理造成影响。可以说技术落后已经成为房企数字化转型的一个瓶颈。

与此同时，物联网、云计算等新一代信息技术在各个行业的应用也在不断加深，并且给房地产数字化转型带来新的动力[2][3]。越来越多的企业开始认识到信息化系统建设的重要性。因此开发一套功能完善、部署灵活的销售管理系统对于提高中小型房企经营效率来说具有十分重要的意义[1][5]。

本文使用Python+Flask框架，采用前后端分离的方式进行设计与实现，创建出一套智能销售管理系统的架构，并且对其实现过程进行了详细的描述。主要涉及房产销售全流程即客户跟进、房源库存控制、折扣审批、签约盖章及收款等环节的工作内容。目的是为了帮助企业把销售活动转移到线上来提升内部协作效率，并且使客户的整个体验从看房到买房变得更好起来。

1.2 国内外研究现状

1.2.1 国外研究现状

欧美等发达国家房地产科技产业起步较早，在经过多年的不断发展之后已经形成了比较成熟的技术体系。从现有的研究来看，国外售楼管理系统主要具有以下几个特点。

在国外系统中一般采用前后端分离加微服务的架构方式，并且使用容器部署来保证高可用性和弹性扩展。该种架构的优势是每个模块可以独立开发、上线，互不影响，在后期维护上也更加方便。

但是也要看到的是，这套方案虽然技术上很先进，但是对运维能力的要求也很高。中小企业如果直接照着模仿的话门槛其实并不低。

再来说说智能应用方面已经发展到相当深的程度了。近些年来人工智能和机器学习在房地产领域的使用早已不是简单的数据记录、查询而已。它们已经开始做预测分析，并且可以自动做出决策。举个例子，有的系统会用AI来评估交易风险，自动生成合规报告；还有的通过自然语言处理技术分析客户反馈，反过来再优化服务质量[6][7]。但是需要注意的是，这些智能化的功能大多面向大型开发企业，专门针对中小型房企的轻量化智能方案比较少。

国外系统非常重视生态集成能力。成功的系统一般都会给第三方服务提供开放的API接口，并且与电子签名、地理信息系统、金融机构等第三方服务商深度整合起来，从而形成一个完整的“服务链条”[3]。但是要考虑到国内政策和市场环境的不同之处，在借鉴国外生态化设计思路的时候还需要做一定的本土化调整。

1.2.2 国内研究现状

国内房地产管理软件市场在政策、市场和技术多重因素驱动下，近年来发展较快，呈现出一些本土化的特征。

政策驱动作用比较明显。这些年国家陆续提出“数字经济”“人工智能+”等发展战略，要求各行各业加快数字化转型步伐。房地产企业也不例外，数字化转型已经进入深水区，售楼系统不再只是个销售工具，而是成了赋能整个业务流程的战略平台[1][2]。受此影响，现在的售楼系统逐渐开始集成合规审查、政策解读这些智能化功能。

移动化和社交化特征比较突出。依托微信等超级App的生态，国内售楼系统在移动端功能开发上做得更深入。同时，针对国内房地产市场“渠道为王”的特点，系统还需要具备渠道管理功能，比如分销商管理、客户判重、佣金计算等本土化难题[8][5]。我们再来分析一下这一部分，它和国外系统的设想方法有很大不同之处。

技术选型上出现多元化趋势。大开发商的企业系统主要使用成熟的Java框架[5]，但是Python技术栈在中小型公

司、创业团队中越来越受欢迎。具体来说，基于Python Flask框架开发的系统已经应用于智慧楼宇管理[9]、运营信息管理[10]等场景，并且用Flask做前后端分离的Web开发也形成了比较成熟的做法[11]。另外，使用Django开发出的住房信息管理系统[12]也可以说明Python这套技术栈可以用于房地产信息化系统的建设当中。

再说数据库性能优化的问题，针对SQL Server这类数据库的优化策略研究[2]、查询负载管理方法[7]等也都给提高系统数据处理效率提供了一些理论上的参考。从信息资源管控平台可视化展示技术研究[13]可以看出，销售数据直观呈现的方法可以有很多种。客户关系管理系统（CRM）是现代企业为了实现对客户的有效管理和营销而创建的一种信息系统。但是由于各种原因，在系统集成以及数据互通方面还有待改进之处。国内销售类系统的功能虽然越来越全了，但是数据孤岛的现象依然存在。系统同内部ERP、财务系统及外部政府平台、银行系统之间存在着数据壁垒，不能很好地实现信息共享和交换，造成数据的一致性、时效性的缺失。另外，标准化产品与企业的个性化需求之间也存在矛盾，深度定制开发成本高、周期长，后期升级维护也比较困难。

总体来看，国内售楼管理系统研究已经取得不少进展，但在轻量化、易扩展、低成本方面仍有改进空间，这也正是本系统想要探索的方向。

1.3 研究内容和目标

本文旨在利用Python Flask框架搭建一套完整的房产销售管理系统，并且把整个销售业务流程都包含进去。具体研究内容如下。

第一部分主要是对中小型房企的销售管理流程进行分析和调研，主要从客户跟进、房源管理、折扣审批、购房合同签订、财务收款等几个方面入手来梳理出系统的各个功能模块。在此基础上画出了系统的整体架构图，包括展示层、服务层和数据持久化层如何分层设计，并且给出了后台管理系统界面以及客户端界面。

在系统设计与开发阶段根据实际销售业务需求，在后台管理端搭建了六个核心模块。客户管理模块负责客户的资料录入、跟进记录的保存和分配工作；房源管理模块对楼盘房源进行资料整理和状态控制的工作；销售业务模块可以发起购房合同，完成折扣逐级审批，进行证件核验等操作的功能实现；财务收款模块用于处理付款信息的输入、到账情况确认以及收据管理等功能的操作；数据统计模块可以执行多维度的数据分析并展现可视化成果；系统权限模块负责用户账号创建、角色分配及日志记录工作。客户端模块给用户提供了在线看房、选房锁房、进度查询，合同和收据下载等服务。

最后就是对整个系统的测试以及优化过程。从功能上讲，需要完成全部的功能单元进行逐一的验证；从流程上来讲，应该保证所有的业务流程都是完整地执行起来的；从性能上讲，则要确保系统具有良好的响应速度。在测试过程中所发现的问题都得到了改进，并且最终使系统稳定可靠地运行起来。

预期达到三个目标：第一，完成一套完整的、稳定的房产销售管理系统，尽量实现客户跟进到财务收款全过程的数字化管理；第二，在实际项目中检验基于Python Flask加Vue前后端分离架构是否适合中小型房企销售系统的开发需求，给同类项目提供技术参考；第三，通过本系统开发实践来提高自己的软件工程能力、数据库设计能力和Web开发能力。

2. 系统开发相关技术

2.1 Python技术

Python这门编程语言是由Guido van Rossum创建的高级解释型语言。开发者一般会认为它语法简单、读起来也

方便。跨平台支持面向对象和函数式等不同的编程范式的使用。可以看见，在Web开发、数据分析、人工智能等领域中，已经有很多成熟的案例应用了Python。再加上其拥有丰富的第三方库生态以及活跃的开源社区，具有很高的开发效率，并且学习门槛低。因此中小型项目选择它的用户数量较多。

就Web开发来说，Python有Django、Flask、Tornado等几个比较成熟的技术框架。开发者可以按照项目的规模和具体的需求来决定使用哪一个。语法简单就是说写代码以及后期的维护都不费劲；但是它强大的数据处理能力又使后续的数据统计和分析功能更加得心应手[10][13]。考虑到这些特点，我认为Python是非常适合作为这套系统后端开发的基础语言。

2.2 Flask框架

Flask是Armin Ronacher用Python写的轻量级Web应用框架。与其他Python的web框架相比它走的是“微内核”路线，核心代码非常简短，只提供Web开发最基础的东西，即路由映射、请求响应处理以及简单的模板引擎。至于项目还需要什么别的功能，开发者可以根据实际需求自由选择第三方库。所以Flask的灵活性和可扩展性都非常强[10][11]。

具体来说Flask的灵活之处表现在哪些方面呢？第一，它的路由系统简单明了，用装饰器就可以把URL和对应的处理函数关联起来，代码读起来也很清晰。第二，它支持多种数据库扩展，比如使用Flask-SQLAlchemy这个扩展可以轻松集成 MySQL、SQLite等关系型数据库的数据持久化操作。第三，它提供了一套统一的扩展接入机制，开发者可以根据具体业务需要随时加入额外模块[9][10]。

那么为什么选择Flask作为后端开发框架呢？主要考虑以下几点。一是轻量又灵活，销售管理系统功能模块多但是各个模块之间逻辑上独立，用Flask可以按照功能边界把它们组织成一个个模块来避免框架本身的负担过重。二是学习门槛低，Flask设计简单直观，开发者上手快、开发效率高。三是能很好地配合前后端分离架构，利用Flask构建出RESTful API，并且能够通过JSON格式同基于Vue的前端应用进行交互[11][15]。再说Flask对ORM支持也很扎实，可以简化数据库操作，减少手写SQL工作量，还能有效防范SQL注入等安全问题[6][7]。综合来看，Flask框架轻量灵活的特点正好满足这套系统后端开发的需求。

2.3 Vue框架

前端框架的选择直接关系到用户交互体验的好坏、开发效率的高低和后期维护成本的大小。目前市面上主流的前端框架有React、Vue和Angular。React主要采用组件化的方式以及虚拟DOM技术来创建一个具有很强交互性的单页应用，但是它的学习曲线比较陡峭。Angular是由Google推出的面向企业级开发的一套框架，功能很全面，但是体积也很大，在中小型项目中可能会显得笨重[11][15]。Vue是尤雨溪创建并维护的一个渐进式JavaScript框架。相比React和Angular来说，Vue在设计上更重视易用性与灵活性的平衡。它采用声明式的渲染方式以及基于组件开发的方式，开发者可以把界面拆分为一个个独立可复用的组件，从而大大方便了团队协作、后期的功能扩展等各方面的工作。Vue具有渐进式的特点，可以按照项目的复杂程度逐步地添加所需功能，并不会被框架本身所限制住[11][13][16]。

回到我们项目实际需求，选择Vue作为前端开发框架的原因有以下几点。第一，学习成本和开发效率。Vue的中文文档很齐全，语法又接近原生HTML、CSS和JavaScript，开发者上手很快。第二，组件化能力。销售管理系统分为后台管理端以及客户端两个部分，包含许多功能模块，使用Vue的组件化开发可以把各个功能封装成独立的组件，在

代码复用率上可以达到很高的水平[5][15]。第三是工具链配套完善。Vue Router负责前端路由的控制，Pinia或Vuex做状态管理的工作，再加上Element Plus等UI库就可以轻松地搭建出统一风格的后台页面了。第四就是和后端技术栈配合得好。Vue通过Axios发送HTTP请求来对接Flask框架开发出来的RESTful API，前后端职责分明、耦合度低[10][11]。

总的来说，由于易用性好、组件化程度高、生态完善以及与Flask良好兼容而被选为这套系统的前端开发工具的Vue框架就是一套比较合适的选择方案，可以很好地满足我们所有的开发需求。

2.4 MySQL数据库

数据库是信息管理的核心基础设施，选什么数据库直接影响到数据存得快不快、查得顺不顺以及后期维护成本的高低。主流的relational数据库有MySQL、Oracle、SQL Server、SQLite这几款，各有各的适用场景。Oracle功能强大但是授权费用高，并且部署要求也较高，在金融或者电信行业的大公司中使用比较普遍。SQL Server和.NET技术栈配合较好，但是跨平台的能力差，而且还有软件授权方面的额外开销[6]。SQLite是轻量级嵌入式数据库，适合手机应用或小型桌面软件，但是在高并发写入场景下性能就会变得很吃紧了，多个用户同时访问Web用时很难撑住。再看MySQL，它是由Oracle维护的一款开源关系型数据库，在功能完整性、性能和成本控制之间取得了不错的平衡。支持标准SQL语法，并且具备事务处理、索引优化以及主从复制等企业级功能。只要在经常查询的字段上建立好索引，单次查询响应时间就可以达到毫秒级别，系统交互就不会卡顿[7][9]。另外MySQL的InnoDB存储引擎可以实现事务和foreign key约束，非常适合对数据一致性有较高要求的场景。

本研究最终选择MySQL作为系统的数据库。第一是成本因素；第二是性能与稳定性。销售管理系统里的客户信息、房源数据、销售记录这些日常业务数据量，MySQL完全能轻松应对。第三，与Python技术栈的兼容性。MySQL提供了丰富的Python连接驱动，可以和Flask框架、SQLAlchemy这类ORM工具无缝对接[9][10]。第四是运维方便。MySQL支持大多数的操作系统，并且安装配置也十分容易，社区也很活跃，遇到问题很容易就能得到解决的办法[11]。综合来看，MySQL具有高性价比、良好的性能以及生态适配能力等特点，为这套系统的数据存储和管理提供了稳定可靠的支持。

2.5前后端分离框架

目前来看，前后端分离已经成为了Web开发的主流标准。它主要就是把一个项目分成前端、后端两个独立的子系统，并且用标准化接口来交换数据。这样一来，技术上就可以解耦了，职责也更明确[11]。以前的传统web开发模式中，前端代码和后端代码常常被紧紧地绑在一起，在页面渲染与业务逻辑之间互相干扰，维护起来比较困难，想要适应不同的设备也很麻烦。而前后端分离的架构正好可以解决这些麻烦。前端负责页面展示、用户交互等工作，并且用Axios等工具发送HTTP请求；后端主要处理业务逻辑以及数据操作工作，对外提供统一API返回JSON格式的数据。只要接口文档定好之后就可以让前后端开发者同时进行开发了，各自独立测试并上线互不影响[11][13]。

从技术上讲，前后端分离有诸多好处。它降低代码耦合度，每个子系统只负责自己那一部分工作，责任界限分明。一套后端API可以同时支持浏览器、手机等不同的终端，适应多设备的便利性不言而喻。团队协作也会更加顺畅，前后端并行开发效率自然就高了。另外，各自独立也方便做性能优化，哪块有瓶颈就单独调哪块。这套系统采用的是前后端分离架构，前端使用Vue框架，后端使用Python Flask框架。前端负责后台管理端和客户端的界面渲染以及用户交互工作；后端主要处理客户管理、房源销售控制、折扣审批、财务收款等核心业务逻辑，并把结果以

JSON格式返回给前端。本人认为该种结构保证了系统的运行速度和可维护性，还为以后的功能扩展和多终端适配留有技术上的余地[11][15]。

3. 系统分析

3.1 可行性分析

可行性分析是系统开发前期的一项重要工作，主要目的是从技术、经济、操作等多个角度评估项目是否具备实施条件，从而为后续的开发决策提供依据。下面从这三个方面对本售楼管理系统进行分析。

3.1.1 技术可行性

本系统采用前后端分离架构，后端使用Python Flask框架，前端使用Vue框架，数据库选用MySQL。这些技术现在都比较成熟，Python语言简洁易用，Flask框架轻量灵活，很适合用来开发中小型Web应用[9][11]。Vue框架上手不难，组件化开发能帮我们把管理端和客户端的界面搭建得比较清晰。MySQL是常用的开源数据库，性能稳定，跟Python配合起来也很顺畅[6][7]。从功能需求来看，这套技术组合完全可以支撑客户管理、房源销控、折扣审批、财务收款等核心模块。再加上这些技术社区活跃，遇到问题基本都能查到解决方案。

3.1.2 经济可行性

本系统的目标用户是中小型房地产企业，成本控制比较重要。开发所用的Python、Flask、Vue、MySQL都是开源技术，不用掏钱买授权；开发工具用PyCharm或VS Code的免费版就行。部署方面可以选云服务器，按需付费，前期投入不大。系统上线后，能把客户跟进、房源销控、认购签约到财务收款这些流程搬到线上，减少人工统计和沟通的工作量，客户的购房体验也会更好一些。

3.1.3 操作可行性

使用本系统的人员包括置业顾问、销售经理、销控专员、财务员、营销总监以及购房客户，不同角色有不同的操作界面和权限。管理端采用常见的后台布局，功能按模块划分，操作流程跟实际业务习惯比较接近，员工简单培训就能上手。客户端界面简洁，客户通过浏览器就能完成在线看房、选房认购、进度查询等操作，不用安装额外软件。再加上操作手册的辅助，学习成本较低。

3.2 系统流程分析

3.2.1 购房资金流转分析

购房资金的流转牵涉到多个环节，并且要符合相关的法律规定。本人开发这套系统的时候对于每一个环节都设计了相应的功能来记录和追踪每笔款项的状态。

先说定金阶段，购房者意向确定之后需要支付一定数额的钱把房源锁住。按照民法典第586条的规定，定金合同自实际交付定金之日起生效，并且定金的金额不能超过主合同标的额的20%。在实际操作当中一般会将定金占房屋总价的1%-5%左右作为支付比例。这笔钱必须存入政府指定的商品房预售资金监管账户内，开发商不得自行收取。为什么要设立这个账户呢？是为了防止出现资金被挪用的情况而保证项目可以正常开展、按时交付。在此阶段系统将会对定金的支付情况进行记录，并且将房源状态由原来的“在售”变为“锁定”，同时标注定金具有法律性质：如果付定金的一方不履行义务，就无权要求返还；收定金的一方如果不履行，则要双倍返还。

接下来是首付和尾款阶段。购房者签正式购房合同之后，按当时的政策规定支付首付款。拿首套房屋来说最低首付比例一般在总价的15%到20%之间。首付款同样需要存入预售资金监管账户里，之前已经交过的定金会自动转为

首付款的一部分。实际上整个购房款都要进入这个监管账户里面。到了这个阶段，系统就会记录首付款到账信息，并且扣除之前所缴纳的定金，计算出接下来还需要支付多少钱。财务人员确认之后，再把房源状态更新为已售，同时生成正式合同给购房者下载。

最后就是按揭贷款阶段了。如果购房者选择贷款买房，银行审批通过后，贷款资金就会直接划到开发商的监管账户中，购房者本人不再需要经过手操作。但是系统要对贷款的申请和放款情况实时进行跟踪记录，记录下各个阶段的状态，“待提交”、“审核中”、“已放款”，等等。贷款到账之后，系统会自动更新收款信息，使财务团队能够及时了解现金流状况。

从定金支付到首付、尾款再到按揭放款这整个资金流动过程都被所覆盖的这套系统包含着诸多重要的资金流转节点。通过款项记录、到账确认和状态更新的方式把资金流动同销售业务联系起来最终汇总到财务收款模块以及数据统计报表当中去，从而实现企业对客户付款进度与整体销售回款情况的全过程跟踪。

3.2. 系统流程分析

客户注册账号、填写好基本信息后，系统就会自动分配一名置业顾问给该客户。客户在客户端浏览房源时看到自己喜欢的就可以加入收藏夹或者通过在线咨询与置业顾问联系。置业顾问管理端可以查看自己分配到的所有客户，并且一边记录跟进情况，一边为客户的意向程度打标签。

等到客户真正下定决心购买的时候，在客户端发起认购申请、选择好房源之后再提交。置业顾问先核实客户信息，再填写折扣申请表和付款方式，把单子送出去审批。系统会根据折扣的幅度来决定审批路径：常规折扣转给销售经理，超额折扣则转给营销总监。审批人查看客户的跟进记录以及房源信息后，再填写审批意见。如果审批通过，则销控专员将房源状态设为“锁定”，系统自动生成电子认购书；不通过的话，单据就退回给置业顾问，让他修改后再提交。

客户在客户端可以随时看到认购进度，到了交定金的环节就线上付款。财务人员确认定金到账后，在系统里记录下定金金额。客户支付首付款之后，财务人员再次检查是否到账，并且销控专员把房源状态从“待售”变为“已售”，此时系统会生成正式的电子合同。客户可以在客户端直接下载合同和发票。若客户选择按揭贷款，置业顾问记录贷款状态，银行放款后财务员确认尾款到账。客户可在客户端随时查看认购、付款、合同等进度，收房前可提交验房申请，系统生成报修单分配给工程部门。

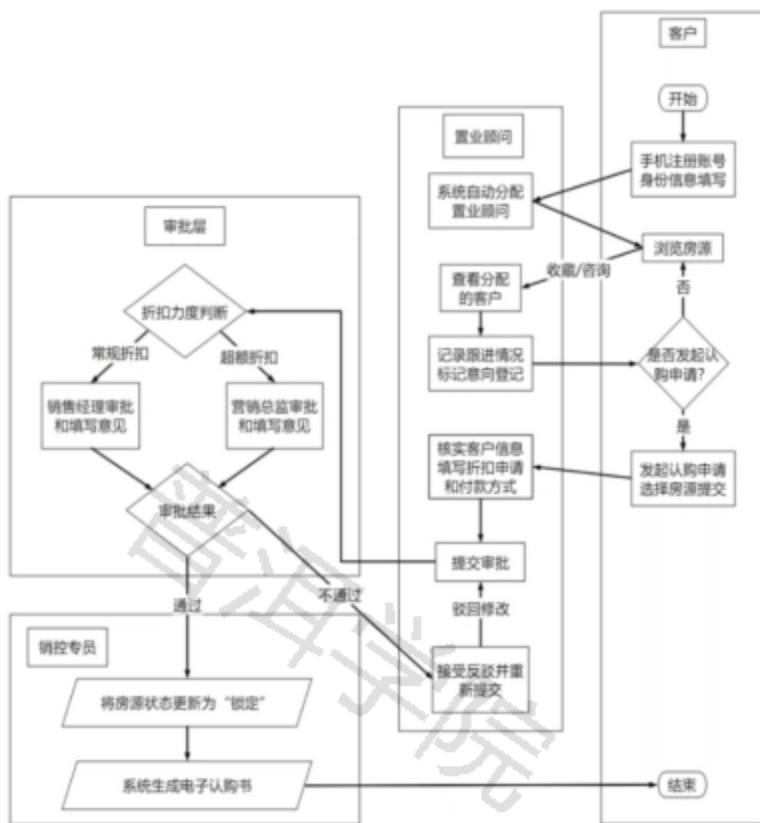


图 2.2-1 系统流程图（1）

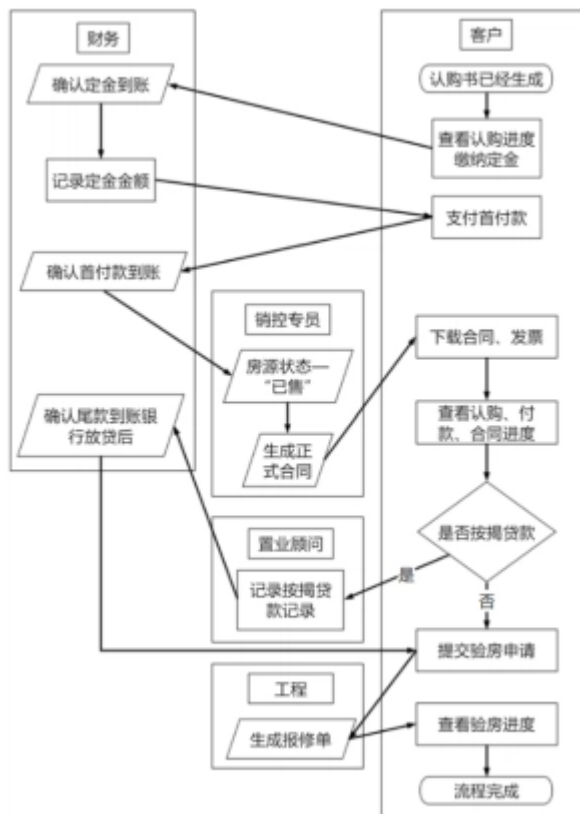


图 2.2-2 系统流程图（2）

4. 系统设计

4.1总体架构设计

本系统采用的是B/S架构也就是浏览器加服务器的方式，前后端是分开的。纵向分为三个层次即展示层、业务逻辑层和数据层。各个层之间用标准接口进行交互，职责划分得很清楚，耦合度也很低。展示层专门处理用户之间的交互以及界面显示；业务逻辑层负责各种核心业务请求的执行；数据层负责数据的持久化存储和管理。简单来说就是各管自己的事物、互不干扰。

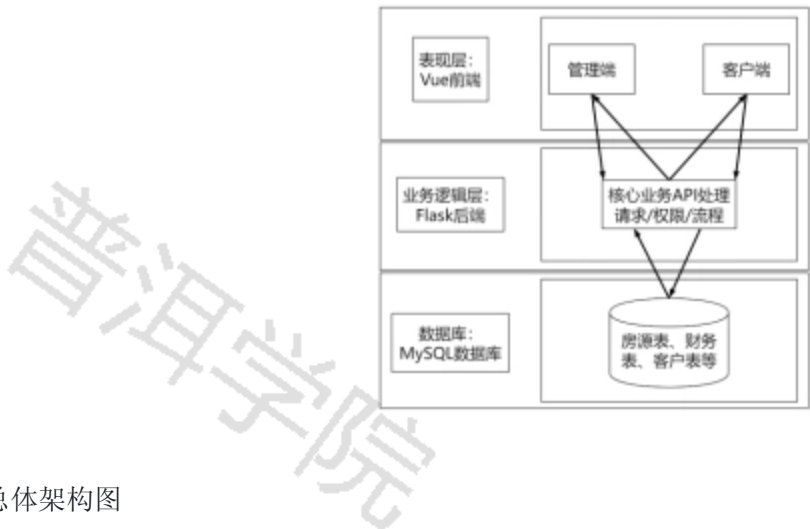


图 4.1 总体架构图

展示层采用的是Vue框架来搭建管理端以及客户端界面。管理端给内部员工使用，用户可以在上面对客户进行管理、房源进行管理、销售业务进行操作、财务收款等。客户端面向购房者提供在线看房、在线咨询、认购申请和进度查询等功能。两个前端系统都用Axios发送HTTP请求调用后端API，服务器返回的数据会动态渲染到页面上。

业务逻辑层采用的是Python Flask框架来搭建的，所有的核心API都在该层里面封装好，用户认证、客户跟进、折扣审批、房源锁定、到账确认以及统计报表等都是在这里进行处理。这一层收到前端发来的请求之后先执行相应的业务规则，再用SQLAlchemy同数据库交互，最后把结果转换成JSON格式返回给前端。

数据层使用MySQL关系型数据库，所有的业务数据都存储在其中，用户表、员工表、楼盘表、房源表、客户表、认购记录表、合同表、收款记录表等。负责数据的持久化存储以及完整性约束，在主键和外键之间建立联系，并且用索引来提高查询速度。

该种三层架构中展示层不能直接访问数据库，所有的数据操作都要经过业务逻辑层。这样做的目的是为了保证系统的安全性的同时也方便后期维护。另外前后端分离的设计方式使得前端与后端可以分别独立开发、测试和部署，团队并行工作起来更加顺畅，日后扩展功能也会更简单。

4.2功能模块设计

本系统根据用户角色和业务流程划分为管理端和客户端两大端侧。管理端面向企业内部员工，包括置业顾问、销售经理、销控专员、财务员、营销总监和系统管理员；客户端面向购房客户。

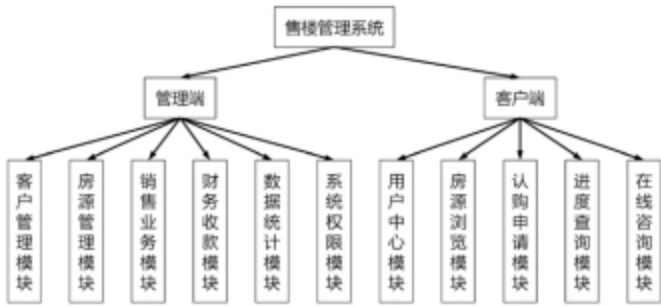


图 4.2 功能模块图

管理端功能模块：

管理端主要面向企业内部员工，覆盖从客户获取、房源管控、销售审批到财务收款、数据统计、系统配置的完整业务链条。各模块功能简要说明如下：

表 4.2-1 管理功能模块表

模块名称	主要功能	操作角色
客户管理模块	客户信息录入、查询、跟进记录维护、客户分配	置业顾问、销售经理
房源管理模块	楼盘及房源信息维护、底价设置、状态管控	销控专员
销售业务模块	认购申请发起、折扣分级审批、认购书审核、房源锁定	置业顾问、销售经理、营销总监
财务收款模块	定金/首付款/按揭款收款确认、票据管理、电子合同生成	财务员
数据统计模块	销售业绩统计、去化率分析、回款进度跟踪、可视化展示	营销总监
系统权限模块	员工账号管理、角色权限配置、操作日志记录	系统管理员

客户端功能模块：

客户端主要面向购房客户，提供从房源浏览、在线咨询、线上认购到进度查询、售后报修的一站式线上服务。各模块功能简要说明如下：

表 4.2-2 客户端功能模块表

模块名称	主要功能	操作角色
用户中心模块	注册、登录、个人信息修改、密码重置	购房客户
房源浏览模块	楼盘介绍、户型图展示、价格查询、房源收藏	
认购申请模块	线上选房、发起认购申请、提交意向	
进度查询模块	认购进度、付款状态、合同生成、报修进展查询	
在线咨询模块	与置业顾问即时沟通、发送文字及图片	

4.3 角色功能设计

网格化社区管理要对责任单元进行精确划分，并且使各个角色之间可以相互配合[17]。本系统的角色权限设计

就借鉴了这个思路，把置业顾问、销控专员这些岗位的职责边界划得很清楚，这样销售活动里大家既能高效分工，又能做细致的衔接。系统一共包含了七种不同的角色，在房产销售过程中对应各个岗位上的人员和客户群体。为了使每个角色的职责范围以及操作权限都明确起来，使用了基于角色的访问控制模型。用户登录时会自动加载与其所用角色相匹配的功能菜单及数据范围。

表 4.3 角色功能设计表

角色	核心功能	权限边界
置业顾问	客户跟进、发起认购申请、填写折扣申请、查看个人业绩	只能操作自己名下的客户和认购单，不能查看其他置业顾问的客户信息
销售经理	审批常规折扣、重新分配公共池客户、查看团队销售数据	可查看本部门所有客户的跟进记录，但不能修改财务数据
销控专员	审核房源信息、上线房源、锁定房源、更新房源状态	拥有房源管理的全部权限，但不能发起认购申请或确认收款
财务人员	确认定金/首付/按揭款到账、生成电子合同、管理票据	只能操作收款确认和票据生成，不能修改房源信息或折扣申请
营销总监	审批超额折扣、查看所有销售数据、统计分析报表	拥有全部数据的查看权限，但不参与具体业务的执行操作
系统管理员	添加员工账号、分配角色权限、维护楼盘项目、查看操作日志	拥有系统配置的最高权限，但不能访问业务数据
客户	注册登录、浏览房源、发起认购申请、查询进度、下载合同、在线咨询、提交验房申请	只能查看和操作与自己账号相关的信息，不能访问内部员工功能

4.4 数据库设计

4.4.1 概念结构设计

在概念结构设计阶段，需要从业务需求中抽象出系统涉及的主要实体及其相互关系。通过对售楼业务流程的分析，识别出用户、员工、楼盘、房源、客户、认购记录、合同、收款记录等核心实体，并明确它们之间的一对一、一对多等关联类型。基于上述分析，绘制了系统部分E-R图：

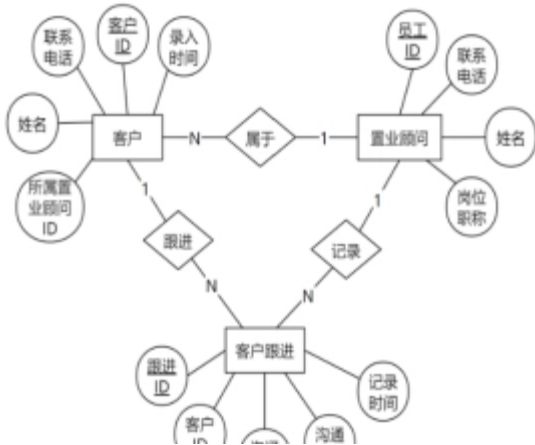


图 4.4-1 客户管理与跟进E-R图

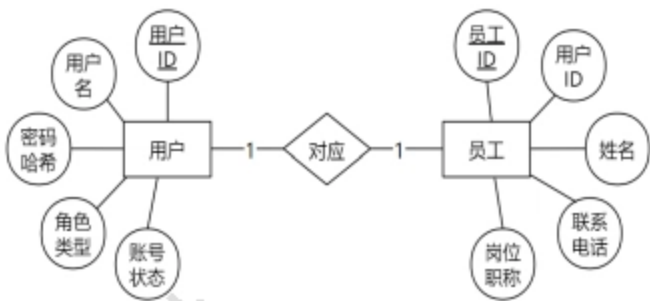


图 4.4-2 系统用户与权限E-R图

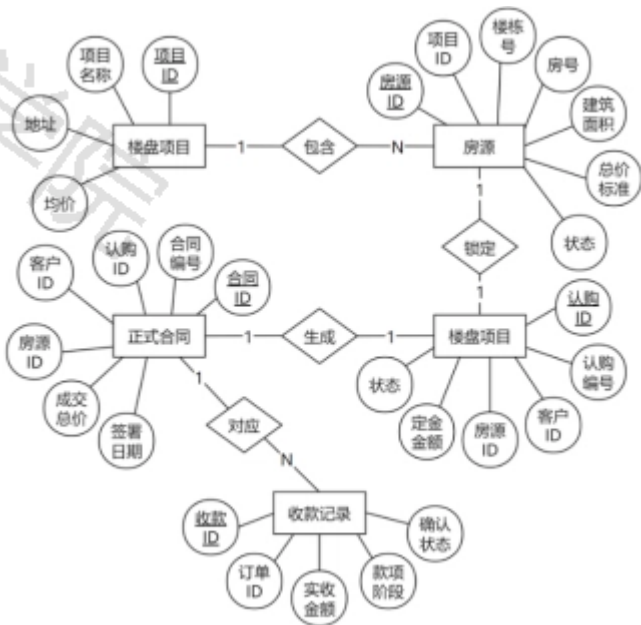


图 4.4-3 销售核心闭环E-R图

4.4.2 逻辑结构设计

设计逻辑结构的时候把E-R图里所有的实体都转化为数据库里的各个表，并且给它们定义好字段名、数据类型、约束条件以及外键之间的关系。所有表都是符合第三范式的，用主键和外键保证了数据的完整性。另外，在经常被查询的字段上建立索引以提高查询速度。系统主要的数据表结构如图4.4-1到4.4-5所示：

表 4.4-1 用户信息表

字段名	数据类型	允许空	备注
id	Integer	否	主键，账号唯一标识
username	String(50)	否	登录名
password_hash	String(128)	否	密码哈希
nickname	String(50)	是	用户中文名称、昵称
role_type	String(20)	否	角色

status	Integer	否	账号状态（1-正常，0-禁用）
--------	---------	---	-----------------

表 4.4-2 员工信息表

字段名	数据类型	允许空	备注
id	Integer	否	主键ID
user_id	Integer	否	关联系统账号（外键）
name	String(50)	否	员工真实姓名
phone	String(20)	否	员工联系电话
position	String(50)	是	岗位职称

表 4.4-3项目楼盘表

字段名	数据类型	允许空	备注
id	Integer	否	项目唯一标识
name	String(100)	否	楼盘名称
address	String(255)	是	项目具体地理位置
average_price	Numeric(10, 2)	否	项目推盘均价

表 4.4-4房源信息表

字段名	数据类型	允许空	备注
id	Integer	否	房源唯一标识
project_id	Integer	否	关联楼盘项目ID
building	String(50)	否	楼栋号
room_number	String(20)	否	具体房号
area	Numeric(10, 2)	否	建筑面积
total_price	Numeric(12, 2)	否	标准总价
status	Integer	否	状态（0-不可售，1-在售，2-已售）

表 4.4-5客户信息表

字段名	数据类型	允许空	备注
id	Integer	否	客户唯一标识
name	String(50)	否	客户姓名
phone	String(20)	否	联系电话
level	String(10)	是	意向等级（A/B/C）

consultant_id	Integer	是	所属置业顾问（外键）
---------------	---------	---	------------

5. 系统实现

5.1 后端核心功能实现

后端基于Python-Flask框架开发，主要实现了身份认证与令牌管理、基于角色的权限控制、认购申请与折扣审批、财务收款与合同生成等核心业务逻辑。

5.1.1 身份认证与令牌机制

本人使用的是基于它的令牌序列化技术，在前后端分离架构下实现无状态的身份认证。具体流程为用户成功登录后，后端会向用户发出一个加密的令牌，并且该令牌中包含有用户的ID和角色信息等。前端拿到这个令牌之后，在每次发送请求的时候就会把其放到Authorization请求头里面去。服务器收到请求之后解析出的令牌就可以完成用户身份识别的功能了。这样服务器不需要保存登录状态，扩展起来也方便很多。

```

1.from itsdangerous import URLSafeTimedSerializer, BadSignature, SignatureExpired
2.from flask import current_app
3.def _serializer():
4.    """初始化序列化工具，利用密钥进行加密"""
5.    return URLSafeTimedSerializer(current_app.config['SECRET_KEY'], salt='re-sales-auth')
6.def create_token(payload: dict) -> str:
7.    """生成加密令牌，包含用户ID、角色等载荷"""
8.    return _serializer().dumps(payload)
9.def decode_token(token: str, max_age_seconds: int = 43200) -> dict:
10.    """解析并校验令牌有效性（默认有效期12小时）"""
11.    return _serializer().loads(token, max_age=max_age_seconds)
```

本人用_serializer根据应用密钥和固定的盐值生成一个序列化器。create_token函数把用户的信息加密成一个令牌，decode_token每次请求的时候解析这个令牌并验证它是否在12小时内有效。得到的载荷之后会进行权限判断。这套方案实现前后端分离模式下的无状态认证，在服务器上也不需要再保存会话信息了。

5.1.2 基于角色的权限访问控制

系统采用RBAC模型，在后端通过装饰器拦截接口，在前端通过路由守卫限制页面访问，确保不同角色只能操作被授权的功能。

（1）后端权限拦截器

```

1.def require_auth(roles=None):
2.    """权限控制装饰器"""
3.    roles = set(roles or [])
4.    def decorator(fn):
5.        @wraps(fn)
```



```

6. def wrapper(*args, **kwargs):
7.     auth = request.headers.get('Authorization', '')
8.     token = auth.split('Bearer ', 1)[1].strip() if 'Bearer ' in auth else None
9.     payload = decode_token(token)
10.    g.user = payload
11.    if roles and payload.get('role') not in roles:
12.        return jsonify({'message': '无权限访问'}), 403
13.    return fn(*args, **kwargs)
14.    return wrapper
15.    return decorator

```

require_auth装饰器接收允许的角色列表。它从请求头提取令牌，解码后获取用户角色，若角色不在授权范围内则返回403错误，否则放行。可灵活挂载在任何需要权限控制的API上，实现接口层的权限隔离。

(2) 前端路由守卫

```

1.router.beforeEach((to) => {
2.    const token = localStorage.getItem('token')
3.    const role = localStorage.getItem('role') || ''
4.    if (to.path !== '/login' && !token) return { path: '/login' }
5.    if (!to.path.startsWith('/mobile')) {
6.        const employeeRoles = ['admin', 'manager', 'controller', 'finance', 'director', 'consultant']
7.        if (!employeeRoles.includes(role)) return { path: '/mobile/home' }
8.    }
9.    if ((to.path === '/rbac' || to.path === '/dict') && role !== 'admin') {
10.        return { path: '/dashboard' }
11.    }
12.    return true
13.})

```

路由守卫在每次页面跳转前执行：首先检查登录状态，未登录则跳转登录页；其次禁止非员工角色访问后台管理页面，自动重定向到移动端首页；最后对系统配置页额外校验管理员角色。前后端双重权限校验，借鉴了社区网格化管理中“精细划分责任单元”的思想。

5.1.3 密码安全哈希

系统不存储用户明文密码，采用加盐哈希算法（SHA-256），确保即使数据库泄露也无法还原原始密码。

```

1.import hashlib
2.def hash_password(plain: str) -> str:
3.    """加盐哈希处理"""

```

```

4. secret = str(current_app.config.get('SECRET_KEY', ''))
5. raw = (secret + '|' + plain).encode('utf-8')
6. return hashlib.sha256(raw).hexdigest()
7. def verify_password(plain: str, password_hash: str) -> bool:
8. """验证明文与哈希值是否匹配"""
9. return hash_password(plain) == password_hash

```

hash_password将应用密钥与明文密码拼接后做SHA-256哈希；verify_password对输入明文执行相同操作并与数据库存储值比对，从而验证密码正确性。该方案有效抵御彩虹表攻击。

5.1.4 财务交易与合同生成

(1) 电子认购书生成

当客户线上抢房成功后，系统自动生成带唯一编号的电子认购书，并锁定房源。

```

1. import uuid
2. from datetime import datetime
3. def generate_subscription(house_id, customer_id):
4. """生成电子认购书逻辑片段"""
5. house = House.query.get(house_id)
6. house.status = 1 # 状态置为“已锁定”
7. sub_no = f"SUB-{datetime.now().strftime('%Y%m%d')}-{str(uuid.uuid4())[8].upper()}"
8. subscription = Subscription(
9.     subscription_no=sub_no,
10.    customer_id=customer_id,
11.    house_id=house_id,
12.    total_price=house.total_price,
13.    status=0, # 草稿/待签约
14.    generation_method=2 # 标识为“线上生成”
15. )
16. db.session.add(subscription)
17. db.session.commit()
18. return subscription

```

该函数首先锁定房源，然后使用UUID生成唯一的认购书编号，最后创建认购记录并提交数据库。认购书的生成意味着线上选房流程的闭环。

(2) 成交签约事务处理

在处理认购签约时，系统需确保“房源状态变更”、“订单创建”、“电子签名存档”三者要么全部成功，要么全部失败，以保证数据强一致性。

```

1.@api.route('/orders', methods=['POST'])
2.@require_auth()
3.def create_order():
4.    data = request.json
5.    house_id = data.get('house_id')
6.    try:
7.        house = House.query.get(house_id)
8.        house.status = 3 # 变为“已售”
9.        new_order = Order(
10.            order_no=generate_order_no(),
11.            house_id=house_id,
12.            real_price=house.total_price,
13.            order_status=1
14.        )
15.        db.session.add(new_order)
16.        db.session.flush() # 获取订单ID
17.        if data.get('signature'):
18.            sig = OrderSignature(order_id=new_order.id, signature=data.get('signature'))
19.            db.session.add(sig)
20.            db.session.commit()
21.        return jsonify({'message': '交易闭环完成'})
22.    except Exception as e:
23.        db.session.rollback()
24.    return jsonify({'message': '交易失败'}), 500

```

该接口在一个事务中完成三项操作：更新房源状态为“已售”、插入交易订单、写入电子签名。任何一步失败都会回滚所有操作，保证数据一致性。财务确认后，客户可在客户端下载电子合同。

5.1.5 折扣业务

售楼业务中常需要申请价格折扣，系统支持置业顾问提交折扣申请，并由销售经理或营销总监分级审批。

(1) 折扣申请接口

```

1.@api.route('/subscriptions/<int:sub_id>/discount-apply', methods=['POST'])
2.@require_auth()
3.def apply_discount(sub_id):
4.    data = request.json
5.    discount = DiscountApproval(

```

```

6. subscription_id=sub_id,
7. apply_discount_rate=data.get('discount_rate'),
8. apply_reason=data.get('reason'),
9. applicant_id=g.user['id'],
10. approval_status=0 # 0-待审核
11. )
12. db.session.add(discount)
13. db.session.commit()
14. return jsonify({'message': '申请已提交'})

```

置业顾问调用该接口，提交认购单的折扣率和申请理由，系统生成一条待审批记录，审批状态初始为0。

(2) 审批处理接口

```

1.@api.route('/discount-approvals/<int:app_id>/approve', methods=['POST'])
2.@require_auth(roles=['manager', 'admin'])
3.def approve_discount(app_id):
4.    data = request.json
5.    approval = DiscountApproval.query.get(app_id)
6.    if data.get('action') == 'approve':
7.        approval.approval_status = 1 # 已通过
8.        sub = Subscription.query.get(approval.subscription_id)
9.        sub.discount_price = sub.total_price * (1 - approval.apply_discount_rate / 100)
10.    else:
11.        approval.approval_status = 2 # 已驳回
12.    db.session.commit()
13.    return jsonify({'message': '审批处理完成'})

```

拥有经理或管理员角色的用户调用该接口进行审批。若通过，则更新审批状态为“已通过”，并重新计算认购书的成交价（成交价=原价×(1-折扣率/100)）；若驳回，状态标记为“已驳回”。审批完成后前端会刷新认购单的价格显示。

5.2 前端界面实现

支持员工和客户通过同一入口登录，系统根据账号角色自动跳转到管理端后台或客户端首页。



图 5.2-1 系统登录界面

置业顾问是唯一能够同时登录管理端和客户端的角色，方便其在后台处理业务的同时，也能从客户视角查看房源展示和认购流程。



图 5.2-2 置业顾问工作空间界面

5.2.1 管理端主要界面

后台管理这块，主要面向公司内部员工。具体涉及六个角色：置业顾问、销售经理、销控专员、财务人员、营销总监，还有系统管理员。系统为他们提供了一系列核心功能，比如客户怎么管、房源怎么控、折扣怎么批、款项怎么收，以及系统本身的配置管理。

(1) 客户管理界面

置业顾问查看分配的客户、填写跟进记录并标记意向等级。



图 5.2-3 置业顾问客户管理界面

销售经理可将公共池客户重新分配给置业顾问。

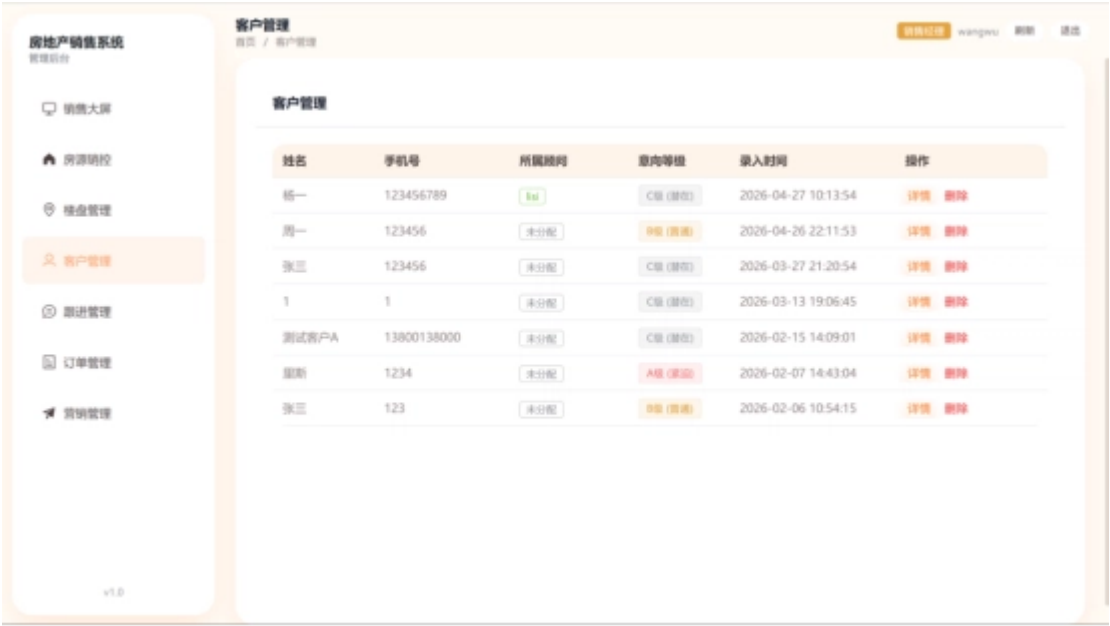


图 5.2-4 销售经理客户管理界面

(2) 房源管理界面

销控专员维护楼盘和房源信息，设置底价，实时管控房源状态（在售/锁定/已售）。



图 5.2-4 房源管理界面



图 5.2-5 楼盘管理界面

(3) 认购审批界面

销售经理或营销总监查看待审批的认购申请，审批通过后自动锁定房源并生成认购书。



图 5.2-6 认购审批界面

(4) 财务收款界面

财务员确认定金、首付款及按揭尾款到账。



图 5.2-7 财务收款界面

(5) 营销活动界面



图 5.2-8 营销活动添加界面



图 5.2-9 营销优惠券添加界面

(6) 系统权限界面

管理员管理员工账号，分配角色权限，并可查看操作日志。



图 5.2-10 系统权限界面

5.2.2 客户端主要界面

(1) 房源浏览界面

客户查看楼盘列表和房源详情，可按户型、面积筛选，支持收藏心仪房源。



图 5.2-11 房源浏览界面

(2) 预约看房界面

客户选择意向房源和期望看房时间，提交预约申请，置业顾问收到后安排接待并确认时间。

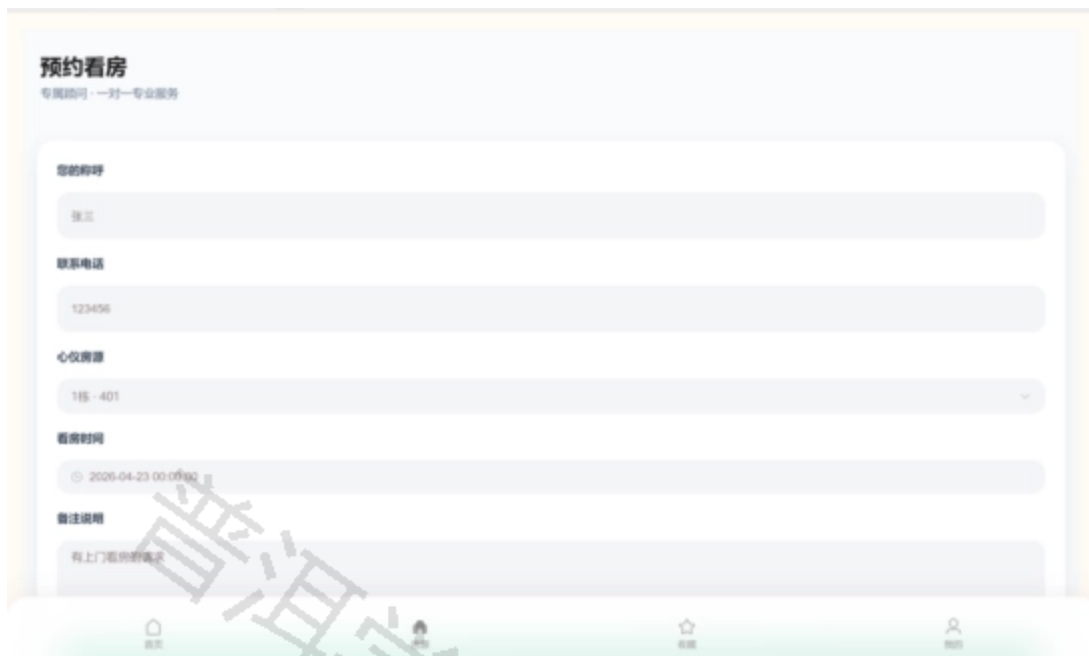


图 5.2-12预约看房界面

(3) 折扣申请界面

客户在选房或认购时可提交折扣申请，填写期望折扣率和申请理由，系统将自动通知置业顾问跟进处理。

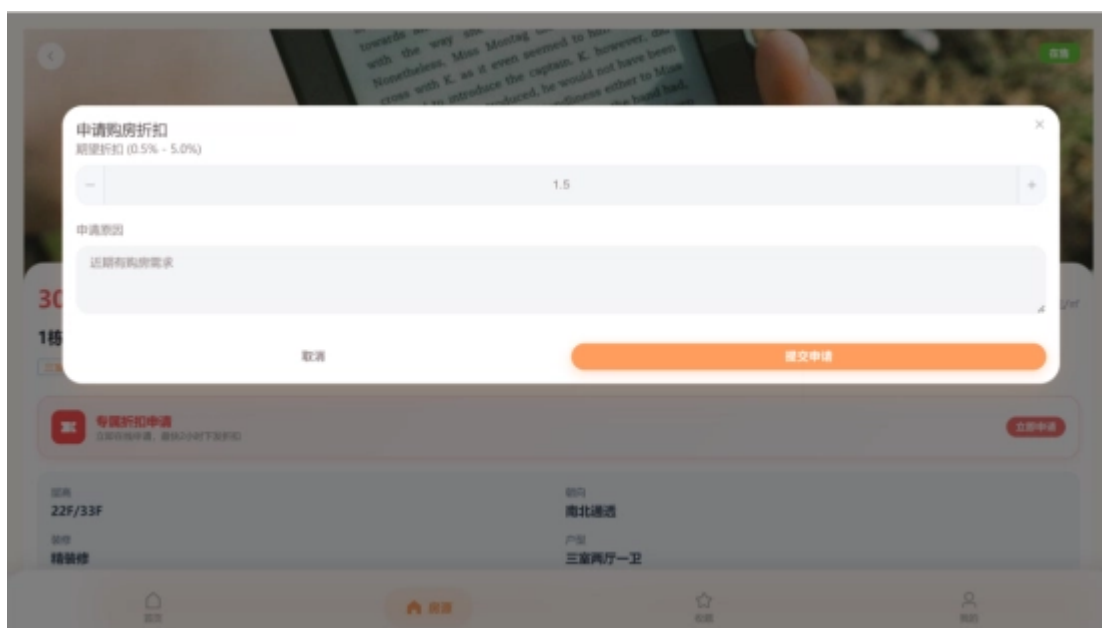


图 5.2-13 折扣申请界面

5.3 关键技术与难点解决

5.3.1 并发场景下的数据一致性

线上开盘或者多人同时认购同一房源的时候会出现多个用户同时锁定一套房的情况造成超卖。为了解决这个问题，系统使用数据库乐观锁机制，在房源表里添加版本号字段，更新房源状态时也验证版本号。若版本号和读取的一致，则更新成功并把版本号加一；如果不一致，则说明数据已经被其他事务修改过，此时需要回滚当前操作，并提示用户刷新后重试。

5.3.2 折扣分级审批的状态流转

折扣审批要经过置业顾问提交申请、销售经理或营销总监分级审批、审批通过以后价格重新计算这几个状态节点。系统用状态机来保证流程的规范化，即申请记录的状态有待审批、已通过、已驳回三种情况，并根据申请的折扣率自动确定相应的审批层级。审批完成后会立即进行认购书成交价的重算工作。

5.3.3 财务确认的事务处理

财务人员确认收款时要更新收款记录的状态为已完成，并把房源状态由锁定变为已售、生成正式的电子合同。这三步必须是原子性的，否则就会出现账实不符的情况。系统用数据库事务包裹这三个操作，任何一个步骤失败都会全部回滚。

5.3.4 权限控制的实现

这套系统有七种不同的用户类型。本人要保证的是只有被授权的人才能访问他们该访问的接口和页面。后端自己写了一个自定义装饰器叫做@require_auth(roles=[...])。它的工作流程是先从请求头中解析JWT token得到当前用户的role信息，如果这个角色不在允许列表里就返回403状态码给对方，“你没有权限”。前端那边也不闲着，本人使用Vue Router路由守卫，在每次跳转页面之前检查本地存储的角色信息。角色不匹配的话就拦下别人进入后台界面。同时，前端菜单也会根据用户所处的身份而变化起来的，每个人看到的是不同的菜单。这样前后夹击之后才会有权限控制上的稳妥。

6. 系统测试

6.1 测试环境与测试方法

这套系统做测试的时候采用的是前后端分离架构。本人把测试环境跟开发环境的配置严格保持一致，这样测出来的结果才真实有效。硬件环境就是普通的办公电脑，软件环境这边则包括前端运行环境、后端服务环境、数据库环境，还有浏览器环境。具体如下所示：

表 6.1 环境配置表

环境	配置
硬件环境	Intel Core i5及以上处理器，8GB运行内存，100GB以上可用存储空间
操作系统	Windows 10 64位操作系统
前端环境	Vue 3、Chrome、Edge浏览器
后端环境	Python3.9+、Flask框架
数据库环境	MySQL8.0关系型数据库
网络环境	局域网环境，保证前后端正常通信

本系统的测试主要使用的是黑盒测试方法。不关心程序内部怎么写，只验证输入、输出是否符合预期。对于管理端以及客户端每一个功能模块来说都会设计出一系列的测试用例来涵盖正常的场景以及异常的情况。

对各个角色权限下的不同操作进行逐一核对。置业顾问可以做哪些事？销售经理可以怎么做呢？销控专员可以做什么，财务可以做什么，本人会检查他们是否都在自己的权限范围内。以置业顾问不能修改房源底价、财务不能批折扣为例说明其限制性。流程测试部分模拟了完整的销售过程，即客户注册→置业顾问跟进→发起认购申请→折

扣审批→财务收款→生成合同的全过程。跑一遍之后就可以检验各个环节的数据流动是否正确、状态更新是否同步。还做了并发测试，比如模拟“多人同时认购同一套房源”的场景，看乐观锁机制能不能正常运作，会不会出现超卖或者状态错乱的问题。

兼容性测试是在主流浏览器上进行的，即Chrome和Edge都做一次完整的前端页面展示以及交互测试。

6.2 功能测试用例与结果分析

针对系统的主要功能模块，本人设计了下面这些测试用例，覆盖了登录认证、客户管理、认购审批、财务收款，还有房源展示这几个板块。每个关键环节都不能漏掉。每个用例都按照测试步骤去执行，然后把实际结果跟预期结果做比对：

表 6.2 功能测试用例表

测试功能	测试步骤	预期结果	实际结果	结论
用户登录	1. 输入正确的用户名和密码 2. 点击登录	成功跳转至对应角色的首页	跳转至首页，显示正确菜单	通过
角色权限隔离	1. 置业顾问登录 2. 尝试访问财务收款页面	提示无权限或页面不可见	页面提示“无访问权限”	通过
客户信息录入	1. 置业顾问进入客户管理 2. 新增客户，填写信息并保存	客户信息保存成功，显示在列表中	列表中出现新客户记录	通过
客户跟进记录	1. 在客户详情中添加跟进记录 2. 填写沟通内容和下次跟进时间	跟进记录保存成功，时间线显示	记录正常显示，时间排序正确	通过
房源状态变更	1. 销控专员将房源状态改为“锁定” 2. 置业顾问查看同一房源	客户端该房源显示“已锁定”，不可认购	房源状态同步更新，无法发起认购	通过
认购申请提交	1. 客户选房并提交认购申请 2. 置业顾问在管理端查看	置业顾问收到待审批申请，房源状态变更为“锁定中”	申请推送至置业顾问，房源锁定	通过
折扣分级审批	1. 置业顾问提交折扣8%的申请 2. 销售经理审批通过	申请状态变为“已通过”，房源锁定	状态变更正确，房源锁定	通过
财务确认收款	1. 财务员查询待确认收款记录 2. 点击确认到账	系统记录收款时间，房源状态变更为“已售”，生成电子合同	收款记录更新，合同可下载	通过
客户端房源浏览	1. 客户登录客户端 2. 查看房源详情	显示户型图、面积、价格等信息	信息完整，图片正常显示	通过
在线咨询	1. 客户发送咨询消息 2. 置业顾问回复	消息实时传递，双方可见	消息发送和接收正常	通过

本人所设计的10个测试用例全部通过了，系统的功能和项目预期是相符合的。在测试过程中也发现一个问题，就是某些页面当窗口变窄的时候会出现错位。后来调整前端样式之后就解决了这个问题。从总体上讲后台管理模

块、客户端模块运行稳定，业务流程正常进行，角色权限控制有效。可以说系统设计目标已经实现。下一步打算优化界面兼容性，并且对异常输入的处理做得更细致一些。

6.3 性能测试与优化措施

6.3.1 性能测试

性能测试结果：针对关键业务场景的测试数据如表所示：

表 6.3-1 性能测试数据表

测试场景	并发/请求量	响应时间/结果	说明
房源列表接口	50并发	平均约150毫秒	建立联合索引后
认购申请接口	10并发	无超卖，少量重试	乐观锁生效
批量数据导出	单次	异步计算完成	避免长时间阻塞
前端首屏加载	—	约1.2秒	开启Gzip+CDN

根据测试结果测了几个关键指标，在50个用户并发访问时房源列表接口平均响应时间为约150毫秒。认购申请用乐观锁保证数据一致性并且可以防止超卖。前端首屏加载时间大约是1.2秒。系统整体上响应速度良好，并发处理能力也比较强，能够很好地满足中小型房企日常运营的需求。

6.3.2 优化措施

针对性能测试中暴露的不足，从数据库、后端接口、前端三个层面采取针对性优化，具体措施及预期效果如表所示：

表 6.3-2 优化措施表

优化层面	具体措施	预期效果
数据库	为status和project_id建立联合索引，为customer_id查询响应时间缩短约25%，减少数据库	
	建立单独索引；将热点房源信息加入Redis缓存	直接访问压力
后端接口	减少联表查询，复杂统计改为异步计算；精简返回字段；	
	分页查询设置合理每页记录数	降低单请求负载，提升并发处理能力
前端	图片资源压缩，路由懒加载，CDN加速静态资源，开启Gzip 首屏加载时间减少约30%，提升用户体	
	验	验

经过上述优化，系统整体响应速度和并发处理能力明显提升，能够稳定支撑中小型房地产企业的售楼业务运行。

7. 总结与展望

7.1 总结

本文使用了Python Flask和Vue框架来开发并实现了一套销售管理系统。为什么要创建这样一个系统呢？主要由于中小型房地产公司销售管理效率低、数据零散、流程不规范等原因造成的。本人采用前后端分离的方式，把后台管理端和客户端的整个过程打通起来，重点对客户管理和房源控制、认购审批以及财务收款这几个关键环节进行设

计工作。通过精细化的角色权限设置来实现置业顾问、销售经理、销控专员、财务人员各自职责，并且能够相互配合协作的工作方式。

系统上线之后，客户的资源得到了很好的整合，在房源状态的实时同步上也达到了一定的规范性。合同和收款记录都可以自动生成，不需要人工干预。本人认为这套系统给企业的内部运营效率带来了一定程度的帮助，客户从看房到买房的过程也变得顺畅起来。开发过程中遇到的一些问题，比如多人同时认购、审批状态如何流转、事务一致性怎样保证等。本人一一应对，用到了乐观锁、状态机以及数据库事务这些策略。整体上达到了预期的目标。这说明Python这套技术栈做企业经营管理类信息系统完全可行，并且可以给其他中小型房企提供一个经济实惠、便于扩展的参考方案。

7.2 系统不足

虽然这套系统基本达到了预期功能的要求，但是本人也意识到它还存在着一些不足之处。并发处理能力比较有限。如果遇到高并发的情况，则响应速度会下降。目前还没有使用缓存或者消息队列来做优化的工作，这是短板。数据分析部分的功能比较简单。只能展示整体销售统计和转化率等简单的图表，没有用到数据挖掘或者趋势预测这样的智能化手段。客户交互体验还有提升的空间。VR看房、在线报修等功能的移动端适配程度不高。再说安全问题。系统目前的安全防护机制比较简单，主要依靠JWT身份认证以及基础的角色权限控制来实现。SQL注入、XSS等常见的Web漏洞还没有做深一层的防御措施。另外数据自动备份、容灾恢复机制还不存在。

7.3 展望

就目前存在的系统短板来说，本人认为可以从以下几个方面进行改进。

技术上可以加一个Redis缓存来保存频繁访问的热点数据。高并发下系统的响应速度就会提高。另外一条路就是往微服务架构方向走，把客户、房源、销售这三个主要模块拆解成一个个独立的小服务。拆解之后系统扩展性及容错性也会得到改善[18][19]。功能上想用大数据分析技术做几个更智能的功能。比如销售趋势走向预测、客户画像精准分析、哪些房源卖不动得提前预警等。管理层得到这些信息后做出的决策会更加可靠[13][20]。

安全以及运维方面要对SQL注入和XSS攻击进行过滤，不能有丝毫遗漏。另外还需要搭建自动备份和容灾恢复方案，保证系统数据的安全性。用户体验移动端适配继续优化，并且加入VR看房、智能客服等功能。购房者体验还可以再上一个台阶。

最后可以推动系统同企业已有的财务系统、项目管理系统对接。

参考文献

- [1]张华, 翟新军, 胥勇, 等. Python在集控大数据应用的研究[J]. 价值工程, 2023, 42(21):84-86.
- [2]郭茜. 房地产企业数字化转型与信息管理[J]. 中国市场, 2025, (16):187-190.
- [3]HUSSAIN A, OBAID J A, TYAGI G, et al. The Next Generation Innovation in IoT and Cloud Computing with Applications [M]. Boca Raton: CRC Press, 2024.
- [4]张辉, 张博, 但扬溪, 等. 基于云计算技术和物联网的大数据集成模型[J]. 电子设计工程, 2025, 33(17):132-135+140.
- [5]师艺玮. 基于SSH框架的房地产销售管理系统的设计与实现[D]. 电子科技大学, 2021.
- [6]LEMIRE E, CARRYL S, SUBRAMANIAN S, et al. The role of housing markets in producing respiratory

health disparities [J]. The Lancet Regional Health – Americas, 2025, 51: 101228.

[7]KRISHAN K, GUPTA G, BHATHAL S G. Query load management: an approach for optimizing database performance [J]. OPSEARCH, 2025. DOI:10.1007/s12597-025-01012-x.

[8]刘晨. 房地产开发公司售楼管理系统的设计与实现[D]. 大连理工大学, 2018.

[9]田胜男. 基于Flask的智能小区物业管理系统设计[与实现[D]. 华东师范大学, 2022.

[10]冷四军. 基于Python Flask的运维信息管理系统设计与实现[J]. 电脑编程技巧与维护, 2021, (05):87-88.

[11]管玲玲, 李浩然. 基于Flask前后端分离Web开发的实现[J]. 电脑编程技巧与维护, 2025, (06):168-170.

[12]XIYA Y, XIANHE L, CHANGPING W, et al. Design and Deployment of Django-based Housing Information Management System [J]. Journal of Physics: Conference Series, 2023, 2425 (1): 012018.

[13]孟显波, 于镇嘉, 宋站丞, 等. 大数据时代的信息资源管控平台可视化展示技术研究[J]. 科技与创新, 2025, (11): 114-117.

[14]尤成武. 客户关系管理在现代企业中的实践研究[J]. 商业2.0, 2025, (22):19-21.

[15]徐厚友, 梁理, 郭昆, 等. 基于Vue的安全评价项目管理系统设计与开发[J]. 工业安全与环保, 2025, 51 (07):66-70.

[16]徐奥辉. SQLServer数据库性能优化策略研究[J]. 信息记录材料, 2020, 21 (12):152-153.

[17]王子仪. 有事大家管 小区很平安[N]. 昆明日报, 2024-07-25 (005). DOI:10.28508/n.cnki.nkmrb.2024.002195.

[18]岳巍. 基于微服务架构的轻量化物联网管理系统设计研究[J]. 信息与电脑, 2025, 37 (15):188-190.

[19]杨俊浩. 微服务架构在企业信息系统构建中的价值分析[J]. 南方农机, 2021, 52 (07):189-190.

[20]代作松, 秦宾, 梁明. 敏捷构建技术的分析与应用拓展[J]. 数字技术与应用, 2025, 43 (06):131-133.

附 录

附录一：系统数据库表

表 附录1 系统数据库表

表名称	物理表名	核心字段
用户信息表	sys_user	id, username, password_hash, nickname, avatar, role_type, status, last_login_time, create_time, update_time
员工信息表	sys_employee	id, user_id, name, phone, email, position, department, supervisor_id, entry_date, status, create_time
项目楼盘表	sys_project	id, name, address, description, average_price, total_houses, sold_houses, status, create_time, update_time
房源信息表	sys_house	id, project_id, building, unit, floor, room_number, layout, area, price_per_meter, total_price, status, orientation, tags, vr_url
客户信息表	sys_customer	id, name, phone, gender, level, source_channel, tags, consultant_id, deal_status, last_follow_time, create_time, remark
		id, customer_id, consultant_id, method, content,

客户跟进表	sys_followup	next_follow_time, create_time id, subscription_no, customer_id, consultant_id,
认购记录表	sys_subscription	total_price, discount_price, deposit_amount, status, is_to_contract, create_time id, contract_no, subscription_id, customer_id, house_id,
正式合同表	sys_contract	final_price, sign_date, record_no, storage_location, status, create_time id, order_id, amount, payment_method, phase, payer_name,
收款记录表	sys_payment	payer_phone, confirm_status, confirm_by, create_time
字典数据表	sys_dict_data	id, type_code, label, value, is_default, sort_order, remark
客户收藏表	sys_favorite	id, customer_id, house_id, create_time
营销活动表	sys_campaign	id, title, type, description, start_date, end_date, budget, responsible_person, status, project_id, create_time

附录二：数据字典补充

表 附录2-1 客户跟进表

字段名	数据类型	允许空	备注
id	Integer	否	跟进ID
customer_id	Integer	否	关联客户ID
method	String(50)	否	沟通方式
content	Text	是	详细沟通纪要
create_time	DateTime	否	记录生成时间

表 附录2-2 认购记录表

字段名	数据类型	允许空	备注
id	Integer	否	认购唯一标识
sub_no	String(50)	否	认购编号
customer_id	Integer	否	认购人ID（外键）
house_id	Integer	否	认购房源ID（外键）
deposit	Numeric(12, 2)	否	缴纳定金金额
status	String(20)	否	认购状态

表 附录2-3 正式合同表

字段名	数据类型	允许空	备注
contract_no	String(50)	否	正式合同编号
subscription_id	Integer	否	关联认购记录（外键）
sign_date	DateTime	否	签署日期
record_no	String(100)	是	网签备案号

表 附录2-4 收款记录表

字段名	数据类型	允许空	备注
id	Integer	否	收款流水主键
order_id	Integer	否	订单 ID
amount	Numeric(12, 2)	否	实收金额
phase	String(50)	否	款项阶段
confirm_status	Integer	否	审核状态（0-未入账，1-已入账）

表 附录2-5 字典数据表

字段名	数据类型	允许空	备注
type_code	String(50)	否	分类编码
label	String(100)	否	显示名称（如：在售）
value	String(100)	否	存储代码值（如：1）

表 附录2-6 客户收藏表

字段名	数据类型	允许空	备注
title	String(100)	否	活动名称
budget	Numeric(12, 2)	是	活动预算成本
start_date	Date	否	活动开始日期
status	Integer	否	进行状态（1, 2, 3）

表 附录2-7 客户收藏表

字段名	数据类型	允许空	备注
customer_id	Integer	否	收藏人ID
house_id	Integer	否	收藏房源ID
create_time	DateTime	否	收藏时间

致 谢

本毕业论文的研究、写作工作已经接近尾声，回顾整个求学过程，在此向所有给予过自己帮助和支持的老师、同学、家人致以由衷的谢意。

首先感谢我的指导老师马航超老师。在论文研究框架确定、内容撰写和修改过程中，马老师给与了必要的指导。系统设计及实现阶段中，马老师就技术选型、架构设计等提出了一些有益的意见，并且帮助我克服了开发当中遇到的一些问题，使系统可以顺利地完成。马老师的严谨治学态度以及扎实的专业知识，在今后的学习工作中给予我很大的启发。

同时感谢普洱学院理工学院各位任课教师。大学期间他们给我的信息管理与信息系统专业课程授课提供了重要的支持，使我在专业知识上有了更深的认识，并且培养了我分析和解决问题的能力，为本次论文的完成打下了较好的专业基础。

须知：

- 报告编号系送检论文检测报告在本系统中的唯一编号。
- 本报告结合AI大模型自动生成，结果仅供参考。
- 报告支持中、英文内容检测。



微信公众号

唯一官网：<https://vpcs.fanyu.com> | 客服邮箱：vpcs@fanyu.com | 客服热线：400-607-5550 | 客服QQ：4006075550